ObRail Europe

Mise en œuvre d’un processus ETL répondant au besoin d’une solution I.A

Équipe

BROCHE Cyprien
DURAND Amandine
LACROIX Lohan
ZAABAT Yousra

Promotion B3 2025-2026
Développeur en Intelligence Artificielle et Data Science

epsi
l’école d’ingénierie informatique

TABLE DES MATIÈRES

Introduction 1
Sources de données 2
Choix techniques 3
Étapes du processus ETL 4
- Phase de collecte 4
- Transformation des données 5
- Phase de chargement 7

Base de données 9
API 11
Nos retours 14

INTRODUCTION

Dans le cadre du débat croissant sur la décarbonation des transports en Europe, la question du report modal de l'avion vers le train s'impose comme un enjeu central des politiques publiques. Le Green Deal européen et le programme TEN-T fixent des objectifs ambitieux de réduction des émissions, dans lesquels le ferroviaire, qu'il soit de jour ou de nuit, joue un rôle stratégique.

C'est dans ce contexte qu'ObRail Europe, observatoire indépendant spécialisé dans la mobilité durable, nous a confié la mission de concevoir et de mettre en œuvre un processus ETL complet permettant de collecter, transformer, stocker et exposer des données fiables sur les dessertes ferroviaires et aériennes à l'échelle européenne. L'objectif final est de constituer un référentiel harmonisé capable d'alimenter des analyses comparatives entre le train et l'avion, notamment en termes d'émissions de CO_2.

Dans ce rapport, nous présentons l'ensemble des choix réalisés tout au long du projet : des sources de données retenues aux décisions d'architecture technique, en passant par les trois phases du pipeline ETL, la conception de la base de données et l'exposition des données via l'API REST. Nous concluons par un retour d'expérience sur les limites rencontrées et les perspectives d'évolution du système.

SOURCES DE DONNEES

Mobility Database

Cette source regroupe les flux GTFS de nombreux opérateurs européens (SNCF, DB, ÖBB, Trenitalia…) et fournit un catalogue complet et à jour des horaires, lignes et arrêts. L'accès se fait via une API REST sécurisée avec token Bearer et est documentée via Swagger UI, ce qui facilite l'automatisation des requêtes. Nous avons choisi cette source pour sa diversité et son volume de données pour les différents pays européens.

Back-on-Track – Night Train Database

Cette base fournit des informations sur les trains de nuit en Europe (lignes, opérateurs, dessertes jour/nuit, distances et parfois émissions). Les données sont disponibles sous forme de fichiers Excel, que nous pouvons télécharger facilement via des scripts simples. Cette source est utile pour compléter les données GTFS avec des informations spécifiques aux trains de nuits.

Ember Energy

Cette source de données fourni des indicateurs d'émissions de CO_2 liés à la production d'électricité à l'échelle des pays. Grâce à l'API REST, nous pouvons récupérer automatiquement l'intensité carbone de l'électricité (CO_2 par kWh produit), par pays (nom et code ISO 3) et par date. Ces données nous permettent donc d'estimer l'empreinte carbone des trajets ferroviaires récupérés en intégrant les différentes spécificités telle que le taux d'électrification de chaque pays.

GeoNames

Cette base mondiale fournit depuis une API les coordonnées (latitude, longitude) des villes, permettant de rapprocher les arrêts issus des flux GTFS (via les *stop_name*) de leurs localisations exactes. Nous l'avons donc choisi pour enrichir et normaliser les informations géographiques des gares et villes, en assurant la cohérence spatiale entre les différentes sources et en facilitant les traitements géographiques nécessaires aux analyses.

OurAirports

Ce site met à disposition des données aéronautiques mondiales, notamment sur les aéroports et leurs caractéristiques. Les datasets sont disponibles via téléchargement direct, que nous pouvons exploiter pour analyser la densité et l'activité aérienne dans différents pays européens. En revanche, la source ne propose pas les « trajets » et « canaux aériens » car se sont des données privées aux compagnies aériennes, et nous n'avons pas trouvé de source open data les diffusant.

ADEME

Enfin, les données de l'ADEME sont utiles pour récupérer l'empreinte carbone des avions en fonction de leur taille et de l'aéroport auquel il est associé. L'accès simple via API permet d'extraire uniquement les données pertinentes pour le projet, apportant une précision supplémentaire sur les émissions liées au transport aérien. Cette source fournit également des données d'émission ferroviaires, mais nous avons choisi de ne pas les utiliser pour cause de leur couverture par pays limité, faisant de grosse approximation.

Sources évaluées mais non retenues

Nous avons étudié d'autres sources lors de notre phase d'exploration, mais nous avons choisi de ne pas les utiliser. Les principales raisons sont les suivantes :

- Couverture géographique trop restreinte (*transport.data.gouv.fr*, *SNCF Open Data*)
- Redondance avec des sources déjà sélectionnées (*Transitland*, *HUSAHUC GTFS France)*
- Format des données inadapté (*Eurostat*, *European Data Portal, UIC)*
- Niveau de mise à jour insuffisant (*European Environment Agency*)

Ces choix ont permis de concentrer notre travail sur des sources fiables, régulièrement mises à jour et couvrant l'ensemble du périmètre européen, tout en garantissant la cohérence et la qualité des données intégrées dans le pipeline ETL.

CHOIX TECHNIQUES

Nous avons développé le projet en **Python**, un langage largement utilisé dans le domaine de la data et de l'intelligence artificielle. Ce choix s'appuie à la fois sur la maîtrise de notre équipe et sur la richesse de son écosystème. De plus, sa syntaxe simple et lisible facilite la collaboration et la maintenance du code, tout en permettant une intégration efficace avec des outils ou librairies externes, telle qu'Apache Spark qui est essentiel pour réaliser notre pipeline ETL.

En effet, **PySpark** est utile pour manipuler et transformer des volumes importants de données à travers des opérations distribuées, ce qui nous a permis de réaliser notre phase de transformation. **Pycountry** nous a aussi servi notamment pour la normalisation des codes pays, facilitant l'intégration de sources hétérogènes.

Pour la collecte des données, nous avons utilisé les librairies **requests** et **aiohttp** pour interroger différentes APIs et sources web, avec une approche synchrone ou asynchrone selon les besoins de performance. **Pandas** a également été utile, mais uniquement pour l'ouverture et la manipulation de fichiers Excel (.xlsx).

Plusieurs librairies complémentaires interviennent pour des aspects techniques transverses : gestion des fichiers et des flux (**os**, **pathlib**, **zipfile**, **BytesIO**), gestion de la configuration (**dotenv**), suivi des tâches et monitoring centralisé (**logging**), manipulation de données (**json**), ou encore gestion du temps et des types (**datetime**, **typing**). Ces outils nous aident à structurer le projet et à en améliorer la robustesse et la maintenabilité.

Ensuite, le stockage des données transformées et agrégées du pipeline ETL dans les DataMarts repose sur **PostgreSQL**. Ce système de gestion de base de données relationnelle est robuste, conforme ACID garantissant l'intégrité des données, et regroupe des fonctionnalités analytiques avancées. Son intégration avec l'écosystème Python est facilitée par l'utilisation de **SQLAlchemy** comme ORM, qui fournit une couche d'abstraction pour mapper les tables de la base en objets Python et simplifier l'exécution des requêtes SQL. Cette combinaison permet ainsi de stocker et d'interroger efficacement les données afin de produire des analyses et des métriques sur les transports.

Pour l'exposition des données via des applications clientes et notre tableau de bord de contrôle, nous avons choisi **FastAPI**, un framework Python moderne, asynchrone et performant. Ce dernier permet de créer des API avec endpoints flexibles, capables d'intégrer facilement des transformations ou calculs complexes avant l'envoie des données. Il offre aussi une documentation interactive automatique, facilitant la compréhension et la maintenance du code, et s'intègre aisément avec l'écosystème Python et l'infrastructure Docker déployée pour le projet. Couplé à PostgreSQL pour le stockage dans les DataMarts, FastAPI assure une exposition fiable, performante et maintenable des données transformées.

Enfin, nous avons développé un dashboard avec le framework open-source **Dash**, développé par Plotly et conçu spécifiquement pour la visualisation de données. Ce choix repose sur son intégration dans l'écosystème Python ainsi que sur sa compatibilité avec notre stack technique existante.

Le déploiement de l'ensemble du projet, incluant l'API, le dashboard, la base de données et le pipeline ETL, se fait avec **Docker**, qui combine simplicité, performance et virtualisation. La conteneurisation permet d'encapsuler chaque composant de l'architecture dans un environnement isolé et homogène, qui garantit une configuration d'exécution identique et reproductible quelle que soit la machine utilisée. Ce point est particulièrement important pour un projet comme le nôtre, où chaque membre de l'équipe dispose de systèmes et de configuration matérielles et logicielles différentes, ce qui peut entrainer des problèmes de dépendances ou de compatibilité.

Cette approche contribue également à structurer le projet sous la forme d'un pipeline ETL complet, autonome et modulaire, dans lequel les différentes étapes d'extraction, de transformation et de chargement des données sont clairement séparées et exécutées de manière cohérente.

ÉTAPES DU PROCESSUS ETL

Afin de structurer et d'automatiser l'intégration des différentes sources de données hétérogènes, nous avons mis en place un pipeline ETL (Extract - Transform - Load). Ce dernier permet de collecter les données provenant des sources mentionnées précédemment, de les nettoyer et de les transformer pour obtenir un modèle de données cohérent et exploitable pour l'analyse et l'exposition via une API.

Comme indiqué dans les choix techniques, l'architecture du pipeline repose sur Python et PySpark et est organisée en plusieurs modules pour garantir la modularité, la maintenabilité et la réutilisabilité du code. Ce découpage permet de relancer indépendamment chaque phase du pipeline : si les règles de nettoyage ou de modélisation évoluent, il est possible de relancer uniquement la transformation sans collecter à nouveau les données.

Le pipeline est orchestré par un module central (*pipeline.py*) qui gère les différentes phases du processus ETL : **extraction des données, transformation et préparation/chargement du modèle de données.**

Nous avons organisé l'architecture du projet autour d'un dossier *src/* contenant les différents modules du pipeline. Un dossier *common/* regroupe les composants partagés par l'ensemble du pipeline, notamment la configuration globale, la gestion des logs, le suivi des performances et la gestion de la session Spark, initialisée une seule fois et partagée entre les différentes phases pour optimiser la consommation mémoire et le temps d'exécution.

Nous avons aussi monté un système de monitoring centralisé et détaillé avec une gestion des logs PySpark, ce qui facilite l'exploitation et la maintenance du pipeline. Il permet de suivre l'exécution des différentes étapes du processus ETL, notamment la durée des traitements et les volumes de données manipulés. Par ailleurs, plusieurs mécanismes d'optimisation ont été intégrés dans les traitements Spark, notamment l'ajustement du nombre de partitions et la matérialisation intermédiaire de certains DataFrames afin de limiter les recomputations et d'améliorer les performances globales du pipeline.

Phase de collecte

La phase d'extraction consiste à récupérer les données depuis les sources identifiées. Étant toutes variées, nous avons développé un système d'extracteur spécifique à chaque source.

Dans un premier temps, nous avons développé un script indépendant pour chaque source afin de tester et valider la méthode de récupération de leurs données. Ainsi, dès qu'ils furent fonctionnels, nous les avons intégrés dans le pipeline, dans un module nommé *extraction/*.

Cette phase repose sur une classe orchestratrice appelée ***RawDataIngestor***, chargée de lancer les différents extracteurs, contenus dans le module *extractors/*. On y retrouve les 6 extracteurs, chacun associé à une source, qui héritent d'une classe abstraite ***BaseExtractor***, fournissant un comportement commun tel que la gestion des erreurs, le suivi des performances et la collecte de métriques.

Exécution en parallèle

Le pipeline devant lancer tous les extracteurs en même temps, la récupération des données était limitée par les opérations réseau (requêtes HTTP et téléchargement). Nous avons donc utilisé le module *Thread Pool Executor* pour optimiser cette phase.

En effet, l'utilisation de thread est adaptée aux tâches *I/O-bound* car elles passent une grande partie du temps à attendre les réponses des serveurs externes sans faire de calcul CPU intensif.

Chaque extracteur implémente une méthode *extract()* qui réalise les opérations spécifiques à la source. Cette dernière est encapsulée dans une méthode *run()* fournie par la classe ***BaseExtractor***, qui ajoute automatiquement :

- le suivi du temps d'exécution
- la gestion des erreurs
- la collecte de métriques (nombre de fichiers téléchargés, taille totale, durée)

Génération du manifeste d'extraction

Une fois la phase d'extraction fini, le script d'orchestration génère un fichier de métadonnées appelé *manifest.json* et le stocke dans le dossier *data/raw/*

Ce fichier est produit par la classe ***MetadataManager*** et contient un résumé de l'exécution pour chaque source avec plusieurs données, telles que le statut de l'extraction, le nombre de fichiers téléchargés, la taille totale des données et la durée de traitement.

Ce mécanisme permet d'assurer la traçabilité et le suivi des exécutions du pipeline.

Stockage des données brutes

Les données collectées sont ensuite automatiquement stockées dans le dossier *data/raw/{source}/* où {source} est le nom de la source.

Ainsi, chaque source possède son propre répertoire pour conserver une séparation claire entre les données. Les fichiers collectés ne sont pas conservés dans leur type d'origine (CSV, JSON, ZIP), mais sont convertis au format *Parquet*, un stockage colonnaire optimisé pour la compression, les performances de lecture et le traitement distribué de grands volumes de données.

Transformation des données

La phase de transformation exploite les fichiers produits lors de la phase d'extraction afin de nettoyer, normaliser et structurer les données, puis de construire un modèle de données cohérent.

Contrairement à l'extraction, cette phase s'exécute de manière séquentielle, puisque certaines transformations dépendent des résultats produits par les étapes précédentes.

L'architecture des dossiers est similaire à celle de la collecte : la logique de transformation est implémentée dans le module *transformation/*. Elle est orchestrée par la classe ***DataTransformer***, qui exécute une série de transformateurs spécialisés. Chaque transformateur est responsable du traitement d'une source de données et hérite de la classe abstraite ***BaseTransformer***, qui fournit des mécanismes communs de gestion des erreurs, de journalisation et de suivi des performances.

Étapes de transformation

Les données ferroviaires issues de Mobility Database sont traitées par le **MobilityDatabaseTransformer**, qui charge et combine les différentes tables au format GTFS (*stops, routes, trips, stop_times, agency, calendar*, etc.). Les flux provenant de plus de 150 réseaux européens sont d'abord harmonisés afin de gérer les différences de schéma entre sources.

Plusieurs opérations de nettoyage et de normalisation sont ensuite appliquées : correction des types de données, suppression des coordonnées invalides (valeurs nulles, vides ou (0,0)), déduplication de trajets provenant de sources multiples, filtrage pour ne conserver que les services ferroviaires pertinents, et normalisation des noms de colonnes. Des traitements temporels et spatiaux complètent ce processus, comme la reconstruction de calendriers de circulation incomplets, le calcul des distances entre arrêts successifs avec des formules géographiques, et le matching des coordonnées pour identifier les correspondances entre gares et arrêts.

Les données complémentaires sont ensuite intégrées et transformées. Le **BackOnTrackTransformer** traite les informations relatives aux trains de nuit européens en combinant les différentes tables GTFS fournies par la source et en les adaptant au modèle utilisé dans le pipeline. Cette étape inclut le nettoyage de valeurs, la normalisation des champs, et l'enrichissement par jointures et complétions internes afin de consolider les informations manquantes.

Par la suite, le **EmberTransformer** et le **AdemeTransformer** préparent les données nécessaires au calcul des facteurs d'émission de CO2 : Ember extrait pour chaque pays l'intensité carbone la plus récente du mix électrique, tandis qu'Ademe normalise les facteurs d'émission fournis par l'ADEME et sélectionne les valeurs pertinentes pour l'analyse.

Enfin, le **OurAirportsTransformer** nettoie les données d'aéroports (suppression des coordonnées invalides, valeurs nulles ou manquantes) et les enrichit en les associant aux villes correspondantes à l'aide des données géographiques de GeoNames, tout en générant les routes aériennes possibles et en réalisant un matching des coordonnées entre aéroports et villes.

L'étape centrale du pipeline est assurée par le **MergeTransformer**, qui fusionne l'ensemble des données et construit le modèle analytique silver sous la forme d'un schéma en étoile.

Pour finir, les données aériennes sont enrichies en ajoutant des clés étrangères vers les villes d'origine et de destination afin d'assurer leur cohérence avec la dimension géographique. Une dernière étape, réalisée par le **StopMatchingTransformer**, identifie les gares et les aéroports situés à moins de 100 km les uns des autres. Cette correspondance spatiale permet d'envisager des combinaisons de trajets train–avion et ouvre la voie à des analyses multimodales des déplacements.

Calcul des facteurs d'emission

Les facteurs d'émission unitaires sont calculés séparément pour les trajets aériens et ferroviaires et stockés dans la table *emission.parquet* en *gCO_2 par passager x km*. Pour connaître les émissions totales d'un trajet, on multiplie simplement ce facteur par la distance, ce qui donne une valeur utilisable pour l'analyse et l'API.

Pour les vols, on récupère les facteurs depuis la base ADEME et on filtre cette dernière pour garder seulement les lignes valides et pertinentes, puis on classe chaque vol selon la taille de l'avion et la distance parcourue. Les facteurs de base sont ensuite combinés pour créer sept facteurs "métier", qui sont appliqués aux routes aériennes grâce à un mapping. Les valeurs finales sont exprimées en gCO_2 / p x km.

Pour les trains, le calcul se base sur la consommation énergétique par type, le taux d'électrification du pays et l'intensité carbone de l'électricité, issue de la source Ember. Les trains 100 % électriques utilisent *consommation × intensité carbone*, tandis que pour les trains partiellement électriques, on fait un calcul pondéré entre électricité et diesel. Chaque couple *pays × type de train* reçoit un facteur, qui est ensuite lié aux trajets par jointure, avec des valeurs de substitution en cas de données manquantes.

Cette méthode permet de comparer les modes de transport de façon cohérente, en utilisant des sources fiables et en gérant les cas où certaines données sont manquantes. En plus des sources déjà mentionnées, nous avons utilisé Eurostat pour les taux d'électrification et des références pour la consommation énergétique des trains et la taille des réseaux ferroviaires européens (UIC EcoPassenger, Wikipedia, Eurostat).

Fonctions utilitaires

La plupart des transformations utilisent des fonctions utilitaires implémentées dans le module *utils/spark_functions.py*, notamment :

- le calcul de distance géographique via la formule de *Haversine*
- la normalisation des noms de villes
- la validation des coordonnées géographiques
- la suppression de doublons dans les trajets

Cette configuration permet d'éviter la duplication de code pour des utilisation similaires.

Résultat de la transformation

Les données transformées sont enregistrées dans le dossier *data/processed/* au format *.parquet* avec compression Snappy, optimisé pour les traitements analytiques avec Spark.

Les principales tables produites sont :

- *train_trip.parquet* : tous les trajets ferroviaires européens (décomposés stop par stop)
- *flight.parquet* : tous les trajets aériens possibles entre aéroports européens
- *localite.parquet* : référentiel unique de toutes les villes européennes présentes dans les données train et avion
- *emission.parquet* : facteurs d'émission unifiés pour le train et l'avion
- *stop_matching.parquet* : associations spatiales gare/aéroport pour trajets multimodaux

Ce modèle de données structuré constitue une base optimisée pour l'analyse, l'exposition via l'API et la création de tableaux de bord.

Phase de chargement

La phase de chargement constitue la dernière étape du pipeline ETL. Elle consiste à transférer les données transformées et agrégées, stockées localement au format Parquet, vers une base de données relationnelle PostgreSQL, afin de les rendre exploitables par l'API et les outils d'analyse.

Le module de chargement est implémenté dans le dossier *src/chargement/*. Il repose sur une architecture modulaire similaire aux autres étapes du pipeline, avec un orchestrateur principal, **DataLoader**, qui coordonne les différentes opérations. Deux composants interviennent dans cette phase : **GoldAggregator**, responsable de la construction des tables analytiques, et **PostgresLoader**, chargé de la gestion du schéma PostgreSQL et de l'insertion des données.

Pour garantir la cohérence et s'assurer que les structures nécessaires sont créées avant l'insertion des données, le processus de chargement est exécuté de manière séquentielle et se décompose en trois étapes.

1. Initialisation du schéma de base de données

Un script SQL prépare la base PostgreSQL en créant la structure nécessaire, incluant les tables, les index et les éventuelles vues. Parmi ces tables figurent la table principale des trajets et une table analytique dédiée à la comparaison des modes de transport. La structure détaillée de ces tables est décrite dans la section *Base de données* de ce rapport, page 7.

2. Construction des tables analytiques

Les fichiers Parquet présents dans le dossier *data/processed/* sont chargés sous forme de DataFrames Spark (niveau ***silver*** du pipeline) et transformés en tables analytiques de niveau ***gold***.

Les trajets ferroviaires et aériens sont d'abord traités séparément, afin d'être enrichis avec des informations géographiques et environnementales issues des tables de localités et de facteurs d'émission carbone. Les deux ensembles de données sont ensuite fusionnés dans une table unique

avec un schéma commun regroupant vingt-sept colonnes. Cette normalisation permet d'intégrer les trajets train et avion dans une même structure de données et facilite les comparaisons entre les différents modes de transport.

Une étape supplémentaire identifie les paires origine-destination pour lesquelles les deux modes de transport sont disponibles et sélectionne la meilleure option de déplacement selon différents critères, notamment les émissions de CO_2.

3. Chargement des données dans PostgreSQL

Une fois les tables analytiques construites, les données sont transférées dans PostgreSQL via le connecteur JDBC de Spark. Au lieu de vider entièrement les tables cibles, le pipeline applique une stratégie de mise à jour : les nouvelles sources sont ajoutées, tandis que les sources déjà présentes sont remplacées pour prendre en compte les éventuelles mises à jour.

Les insertions sont réalisées par lots pour améliorer les performances du chargement. À l'issue de cette étape, deux tables principales sont disponibles dans la base de données : *gold_routes* et *gold_compare_best*. Ces dernières sont aussi conservées au format Parquet dans le dossier *data/gold/*, facilitant des analyses complémentaires et le débogage du pipeline.

Cette stratégie de mise à jour incrémentale permet de conserver les données existantes tout en intégrant les nouvelles sources ou leurs évolutions. En revanche, ces opérations sur de grands volumes de données entraînent des temps de chargement plus importants.

BASE DE DONNEES

Architecture

La base PostgreSQL complétée constitue la couche de persistance principale du système et sert de source de données pour l'API ainsi que pour les outils d'analyse et de visualisation.

Le modèle de données final repose sur deux tables principales du niveau gold, optimisées pour l'exploration et l'exploitation applicative :

- *gold_routes*, qui regroupe l'ensemble des trajets ferroviaires et aériens ;
- *gold_compare_best*, qui contient la meilleure option de transport entre deux villes selon différents critères.

Table *gold_routes*

Cette table constitue la table centrale du modèle. Elle contient environ 93 millions de lignes et regroupe l'ensemble des trajets train et avion dans un schéma unifié, permettant de comparer directement les deux modes de transport.

La table est partitionnée via un type LIST sur la colonne *departure_country* avec 30 partitions pour les principaux pays européens et une partition par défaut (XX) pour les enregistrements hors pays définis. Cette stratégie réduit la taille des ensembles de données parcourus lors des requêtes filtrant sur le pays de départ et améliorent les performances globales du système

via un type LIST afin d'optimiser les performances des requêtes géographiques.

Les colonnes principales de la table incluent :

- des **identifiants et métadonnées de trajet** (*source*, *trip_id*, *mode*, *route_id*, *service_id*) ;
- des **informations d'agence et de service** (*agency_name*, *agency_timezone*) ;
- les **données de départ et d'arrivée** (gare ou aéroport, ville, pays, horaires) ;
- des **informations temporelles** relatives au calendrier de circulation (*service_start_date*, *service_end_date*, *days_of_week*) ;
- **des métriques de trajet**, incluant la distance et les émissions de CO_2 par passager. Le facteur d'émission est exprimé en **gCO_2 par passager.km**, et les émissions d'un trajet sont obtenues en multipliant ce facteur par la distance parcourue.

Cette structure unifiée permet d'interroger les trajets ferroviaires et aériens au sein d'une même table et simplifie les requêtes de l'API.

Table *gold_compare_best*

Cette table analytique dérivée contient environ 90 millions de lignes et permet de comparer les différents modes de transport. Chaque enregistrement correspond à une paire origine-destination pour laquelle au moins une alternative train et avion existe.

Pour chaque paire de villes, la table stocke :

- les métriques du trajet ferroviaire (durée, distance, émissions) ;
- les métriques du trajet aérien correspondant ;
- le **mode de transport optimal** selon les critères définis dans le pipeline.

L'API l'utilise notamment pour fournir rapidement des comparaisons entre les différents modes de déplacement.

Relations et intégrité des données

Le modèle ne repose pas sur des clés étrangères explicites entre les tables. Nous avons fait ce choix pour privilégier les performances sur de très grands volumes de données et simplifier les opérations d'insertion lors du chargement.

Les relations logiques entre les données sont maintenues au niveau du pipeline de transformation et de l'application plutôt qu'au niveau du moteur de base de données.

Indexation et optimisation des requêtes

Pour optimiser les requêtes les plus fréquentes exposées par l'API, nous avons mis en place une stratégie d'indexation post-chargement (script *post_load.sql*) sur les tables analytiques.

Index principaux :

- **BTREE,** type d'index standard qui accélère les recherches par valeur exacte ou par intervalle (ex. recherche par ville de départ/arrivée ou par identifiant de trajet)
- **GIN trigram (pg_trgm)**, index spécialisé pour les recherches de texte partiel avec ILIKE '%...%', utile pour l'autocomplétion et les filtres sur les noms de villes, gares ou agences

Un **ANALYZE** est exécuté après le bulk load pour mettre à jour les statistiques des tables dans PostgreSQL, ce qui permet au moteur de générer des plans d'exécution plus efficaces.

Ces optimisations bénéficient directement aux endpoints */routes*, */routes/search*, */routes/{trip_id}*, */compare*, */carbon/trip/{trip_id}*, */carbon/estimate*, */carbon/ranking* et */carbon/factors*, améliorant les performances des requêtes liées à la consultation des trajets, à la comparaison des modes de transport et aux analyses environnementales.

API

Architecture

L'API que nous avons développée constitue la couche d'exposition des données et permet aux applications clientes d'interroger la base de données PostgreSQL de manière structurée et sécurisée.

L'API repose sur une architecture REST et expose l'ensemble de ses endpoints sous le préfixe : */api/v1*. Toutes les réponses sont renvoyées au format JSON et suivent une structure homogène contenant notamment le statut de la requête, les informations de pagination et les données retournées.

La pagination est appliquée par défaut pour limiter la taille des réponses et optimiser les performances. Les paramètres *page* et *page_size* permettent de naviguer dans les résultats, avec une taille maximale de page fixée à 500 enregistrements. L'API interagit directement avec les tables analytiques *gold_routes* et *gold_compare_best*, spécifiées dans la partie *Base de données*.

Organisation des endpoints

Les endpoints de l'API sont organisés en plusieurs catégories correspondant aux différents usages des données.

Import et orchestration du pipeline

Certains endpoints permettent de déclencher les différentes étapes du pipeline ETL directement via l'API. Ils renvoient un compte-rendu détaillé de l'opération, comme cité en page 3 de ce rapport.

Par exemple :

- POST */import/routes/train* : import des trajets ferroviaires (données GTFS et BackOnTrack)
- POST */import/routes/flight* : import des trajets aériens, calcul des distances et émissions CO_2
- POST */import/full* : exécution complète du pipeline ETL

Consultation des données

La consultation des données constitue l'usage principal de l'API. Les endpoints correspondants permettent d'interroger les tables analytiques produites par le pipeline.

Les endpoints les plus importants sont :

- GET */routes* : consultation paginée de la table *gold_routes*
- GET */routes/download* : export CSV des trajets
- GET */compare* : consultation paginée de la table *gold_compare_best*

Ces endpoints acceptent différents paramètres de filtrage, comme la ville de départ, le pays, le mode de transport, la distance, les émissions de CO_2, etc. pour réaliser des requêtes ciblées.

Recherche et analyse

L'API propose également plusieurs endpoints dédiés à la recherche de trajets et à l'analyse des données. Par exemple :

- GET */routes/search* : recherche de trajets ferroviaires entre deux villes ou deux gares
- GET */carbon/trip/{trip_id}* : calcul du bilan carbone d'un trajet spécifique
- GET */carbon/estimate* : estimation des émissions de CO_2 pour un trajet entre deux villes

D'autres endpoints permettent de produire des analyses statistiques sur les données, comme la distribution des distances ou les émissions moyennes par tranche de distance.

Tests réalisés

Nous avons réalisé plusieurs tests pour vérifier le bon fonctionnement de l'API et sa capacité à traiter des volumes de données importants. Les tests ont notamment porté sur :

- la validité des réponses JSON retournées par les endpoints
- la gestion de la pagination et des paramètres de filtrage
- les performances des requêtes sur de grands volumes de donnée
- la cohérence des résultats retournés avec les données présentes dans la base PostgreSQL

Nous avons aussi effectué des tests de charge, ici pour s'assurer que l'API est capable de traiter un grand nombre de requêtes. Les résultats montrent que l'API peut traiter jusqu'à **100 requêtes par minute**, représentant environ **500 000 lignes de données consultées par minute**, sans dégradation notable des performances.

Sécurité

Pour protéger l'API et la base de données, nous avons mis en place plusieurs mécanismes de sécurité. Tout d'abord, les requêtes SQL utilisées par l'API reposent sur des paramètres liés (parameter binding) et sur une liste blanche des colonnes autorisées pour le tri. Cette approche garantit que les entrées fournies par les utilisateurs ne peuvent pas être interprétées comme du code SQL, ce qui empêche les attaques par injection et évite toute corruption potentielle des données.

Nous avons aussi intégré un système de limitation de débit (rate limiting) afin de prévenir les attaques de type déni de service ou les tentatives d'extraction massive de données. Chaque adresse IP est temporairement enregistrée lors de l'exécution des requêtes et le nombre de requêtes autorisées est limité à 100 requêtes par minute.

Enfin, l'endpoint */import* (POST), qui déclenche le pipeline ETL, nécessite une authentification via un token d'accès. Ce dernier est défini par la variable d'environnement API_IMPORT_TOKEN et permet de restreindre l'accès aux utilisateurs autorisés.

TABLEAU DE BORD

Le tableau de bord ObRail Europe est un livrable fonctionnel répondant à l'exigence du cahier des charges de proposer un outil de contrôle et de visualisation des données, accessible à tout public. Il offre une interface interactive permettant d'explorer les routes ferroviaires européennes, de comparer les émissions CO_2 entre le train et l'avion, d'analyser la distribution des trains de jour et de nuit, et de contrôler la qualité des données agrégées par le pipeline ETL.

Architecture technique

Le dashboard est développé en Python avec le framework **Dash** (Plotly), selon un pattern multi-pages : chaque écran est un module Python indépendant qui déclare sa route via *dash.register_page()*. Le routage est géré côté client (sans rechargement de page), et la sidebar de navigation reste persistante entre les écrans pour assurer un repère visuel stable.

Stratégie de cache

Un système de cache basé sur **diskcache** est intégré de manière transparente dans le client API. Chaque réponse est mise en cache avec un TTL calibré selon la volatilité de la donnée : 24 h pour les KPI globaux (très stables entre deux cycles ETL), 30 min pour les référentiels, 15 min pour les agrégats carbone et jour/nuit, et 5 min pour les résultats de recherche et de pagination interactive. Le cache est partagé avec le gestionnaire de callbacks asynchrones de Dash (*DiskcacheManager*), ce qui permet aux opérations longues (exports CSV de grands volumes) de s'exécuter en arrière-plan sans bloquer l'interface. Un bouton de vidage manuel du cache est exposé dans la sidebar pour faciliter la maintenance.

Pages et fonctionnalités

Le dashboard est articulé en sept pages, chacune consumée directement depuis l'API */api/v1* via des appels parallèles au chargement de la page.

- **Vue d'ensemble** : indicateurs globaux (nombre de routes, pays couverts, part des trains de nuit, CO_2 moyen, complétude des données), top 15 opérateurs, courbe CO_2 par tranche de distance et donut jour/nuit.
- **Routes Explorer** : consultation paginée côté serveur de *gold_routes* avec filtres affichage du détail des segments d'un trip sélectionné, et export CSV de la page courante.
- **Analyse Carbone** : estimateur d'émissions CO_2 à la demande pour n'importe quelle paire O/D (appel à */carbon/estimate*), classement des corridors par économie CO_2 train vs vol (top 15), et carte choroplèthe des facteurs d'émission ferroviaires par pays.
- **Analyse Jour / Nuit** : KPI synthétiques, histogramme empilé de la couverture jour/nuit par pays, émissions moyennes comparées, et comparateur O/D interactif.
- **Qualité des données** : Il expose la complétude globale (jauge), la complétude par colonne (barres horizontales), la couverture géographique (carte choroplèthe), les top villes desservies, la distribution des jours de service, et la balance jour/nuit par pays.
- **Référentiel** : exploration des villes, gares ferroviaires et aéroports via trois onglets distincts. Chaque onglet propose des filtres (pays, recherche textuelle, présence de gare/aéroport) et une table paginée côté serveur avec export CSV.
- **Data** : accès analyste aux deux tables gold avec filtres avancés (mode, pays, ville, gare, opérateur, type de route, plages de distance/CO_2, dates de service, jours de la semaine), tri serveur, pagination et export CSV. La table de comparaison train/vol est également consultable avec filtres et tri.

Design system et accessibilité

Le dashboard est responsive : la sidebar se réduit à une barre d'icônes sur mobile et les grilles se reconfigurent en colonne unique en dessous de 768 px. L'application est lancée en développement via *python -m dashboard.app* et en production via Gunicorn dans le conteneur Docker, en utilisant le point d'entrée WSGI *app.server* exposé par Dash.

NOS RETOURS

Limites identifiées

Malgré la mise en place d'un environnement Docker permettant d'homogénéiser l'exécution du pipeline, certaines différences matérielles entre nos machines de développement (mémoire vive, nombre de cœurs CPU ou espace disque) ont entraîné des variations importantes dans les temps d'exécution. Dans certains cas, il a fallu trouver et ajuster certains paramètres de configuration de Spark pour adapter le pipeline aux ressources disponibles.

Autre point, le volume important de données traitées par le pipeline entraîne des temps d'exécution relativement élevés. Dans sa version optimisée, l'exécution complète du pipeline nécessite environ 1 h 30 sur une machine disposant de ressources matérielles importantes et d'une connexion réseau stable. Sur des machines moins puissantes, ce temps est sensiblement plus long, ce qui a représenté un défi lors du développement et des phases de test.

Enfin, l'intégration de sources de données hétérogènes était une difficulté supplémentaire. Les fichiers utilisés présentent des structures très différentes, tant au niveau des noms de colonnes que des formats de valeurs ou de l'organisation des fichiers. Nous avons dû passer par une phase d'analyse exploratoire approfondie ainsi qu'un travail progressif de normalisation pour harmoniser les données au sein d'un modèle commun.

Pistes d'amélioration

Plusieurs améliorations pourraient être envisagées afin de renforcer la robustesse et l'évolutivité du système. Premièrement, la mise en place d'une suite de tests automatisés permettrait d'améliorer la fiabilité du pipeline. Des tests unitaires seraient utiles pour valider individuellement les différents modules de transformation, tandis que des tests d'intégration permettraient de vérifier le bon fonctionnement du pipeline complet sur des jeux de données de test.

Une autre piste d'amélioration concernerait l'orchestration automatisée du pipeline. Dans la version actuelle, l'exécution du processus ETL est déclenchée manuellement. L'intégration d'un outil d'orchestration dédié, comme un planificateur de tâches ou une plateforme de gestion de workflows de données, permettrait d'automatiser l'exécution du pipeline, de gérer les dépendances entre les étapes et de faciliter la reprise en cas d'échec.

Enfin, nous pourrions envisager certaines optimisations supplémentaires pour améliorer la scalabilité du pipeline, par exemple en mettant en place une stratégie de chargement incrémental. Actuellement, l'ensemble des données est retraité à chaque exécution, ce qui entraîne des temps de calcul relativement importants compte tenu du volume de données traité. Une approche incrémentale permettrait de ne traiter que les nouvelles données ou les données modifiées depuis la dernière exécution du pipeline, réduisant ainsi significativement les temps de traitement.